

```

1  K=kernel
2  U=user
3
4  OBJS = \
5      $K/entry.o \
6      $K/start.o \
7      $K/console.o \
8      $K/printk.o \
9      $K/uart.o \
10     $K/kalloc.o \
11     $K/spinlock.o \
12     $K/string.o \
13     $K/main.o \
14     $K/vm.o \
15     $K/proc.o \
16     $K/shm.o \
17     $K/semaphore.o \
18     $K/swtch.o \
19     $K/trampoline.o \
20     $K/trap.o \
21     $K/syscall.o \
22     $K/sysproc.o \
23     $K/bio.o \
24     $K/fs.o \
25     $K/log.o \
26     $K/sleeplock.o \
27     $K/file.o \
28     $K/pipe.o \
29     $K/exec.o \
30     $K/sysfile.o \
31     $K/kernelvec.o \
32     $K/plic.o \
33     $K/virtio_disk.o
34
35 # riscv64-unknown-elf- or riscv64-linux-gnu-
36 # perhaps in /opt/riscv/bin
37 #TOOLPREFIX =
38
39 # Try to infer the correct TOOLPREFIX if not set
40 ifndef TOOLPREFIX
41 TOOLPREFIX := $(shell if riscv64-unknown-elf-objdump -i 2>&1 | grep 'elf64-big' >/dev/null 2>&1; \
42 then echo 'riscv64-unknown-elf-'; \
43 elif riscv64-elf-objdump -i 2>&1 | grep 'elf64-big' >/dev/null 2>&1; \
44 then echo 'riscv64-elf-'; \
45 elif riscv64-none-elf-objdump -i 2>&1 | grep 'elf64-big' >/dev/null 2>&1; \
46 then echo 'riscv64-none-elf-'; \
47 elif riscv64-linux-gnu-objdump -i 2>&1 | grep 'elf64-big' >/dev/null 2>&1; \
48 then echo 'riscv64-linux-gnu-'; \
49 elif riscv64-unknown-linux-gnu-objdump -i 2>&1 | grep 'elf64-big' >/dev/null 2>&1; \
50 then echo 'riscv64-unknown-linux-gnu-'; \
51 else echo "***" 1>&2; \
52 echo "*** Error: Couldn't find a riscv64 version of GCC/binutils." 1>&2; \
53 echo "*** To turn off this error, run 'gmake TOOLPREFIX= ...'." 1>&2; \
54 echo "***" 1>&2; exit 1; fi)
55 endif
56
57 QEMU = qemu-system-riscv64

```

```

58 MIN_QEMU_VERSION = 7.2
59
60 CC = $(TOOLPREFIX)gcc
61 LD = $(TOOLPREFIX)ld
62 OBJCOPY = $(TOOLPREFIX)objcopy
63 OBJDUMP = $(TOOLPREFIX)objdump
64
65 # Deterministic builds.
66 DETFLAGS = -ffile-prefix-map=$(CURDIR)=.
67
68 CFLAGS = -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -ggdb -gdwarf-2
69 CFLAGS += $(DETFLAGS)
70 CFLAGS += -march=rv64gc
71 CFLAGS += -std=gnu99
72 CFLAGS += -MD
73 CFLAGS += -mmodel=medany
74 CFLAGS += -ffreestanding
75 CFLAGS += -fno-common -nostdlib
76 CFLAGS += -fno-builtin-strncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin-memset
77 CFLAGS += -fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-bzero
78 CFLAGS += -fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc
79 CFLAGS += -fno-builtin-free
80 CFLAGS += -fno-builtin-memcpy -Wno-main
81 CFLAGS += -fno-builtin-printf -fno-builtin-fprintf -fno-builtin-vprintf
82 CFLAGS += -I.
83 CFLAGS += $(shell $(CC) -fno-stack-protector -E -x c /dev/null >/dev/null 2>&1 && echo -fno-stack-protector)
84
85 # Disable PIE when possible (for Ubuntu 16.10 toolchain)
86 ifneq ($(shell $(CC) -dumpspecs 2>/dev/null | grep -e '^[^]no-pie'),)

```

```

87 CFLAGS += -fno-pie -no-pie
88 endif
89 ifneq ($(shell $(CC) -dumpspecs 2>/dev/null | grep -e '^[^]nopie'),)
90 CFLAGS += -fno-pie -nopie
91 endif
92
93 LDFLAGS = -z max-page-size=4096
94
95 $K/kernel: $(OBJS) $K/kernel.ld
96     $(LD) $(LDFLAGS) -T $K/kernel.ld -o $K/kernel $(OBJS)
97     $(OBJDUMP) -S $K/kernel > $K/kernel.asm
98     $(OBJDUMP) -t $K/kernel | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$$/d' > $K/kernel.sym
99
100 $K/%.o: $K/%.S
101     $(CC) -march=rv64gc -g $(DETFLAGS) -c -o $@ $<
102
103 tags: $(OBJS)
104     etags kernel/*.S kernel/*.c
105
106 ULIB = $U/ulib.o $U/usys.o $U/printf.o $U/umalloc.o
107
108 _%: %.o $(ULIB) $U/user.ld
109     $(LD) $(LDFLAGS) -T $U/user.ld -o $@ $< $(ULIB)
110     $(OBJDUMP) -S $@ > $.asm
111     $(OBJDUMP) -t $@ | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$$/d' > $.sym
112
113 $U/usys.S : $U/usys.pl
114     perl $U/usys.pl > $U/usys.S
115
116 $U/usys.o : $U/usys.S

```

```

116 $U/usys.o : $U/usys.S
117     $(CC) $(CFLAGS) -c -o $U/usys.o $U/usys.S
118
119 $U/_forktest: $U/forktest.o $(ULIB)
120     # forktest has less library code linked in - needs to be small
121     # in order to be able to max out the proc table.
122     $(LD) $(LDFLAGS) -N -e main -Ttext 0 -o $U/_forktest $U/forktest.o $U/ulib.o $U/usys.o
123     $(OBJDUMP) -S $U/_forktest > $U/forktest.asm
124
125 mkfs/mkfs: mkfs/mkfs.c $K/fs.h $K/param.h
126     gcc -Wno-unknown-attributes -I. -o mkfs/mkfs mkfs/mkfs.c
127
128 # Prevent deletion of intermediate files, e.g. cat.o, after first build, so
129 # that disk image changes after first build are persistent until clean. More
130 # details:
131 # http://www.gnu.org/software/make/manual/html_node/Chained-Rules.html
132 .PRECIOUS: %.o
133
134 UPROGS=\
135     $U/_cat\
136     $U/_echo\
137     $U/_forktest\
138     $U/_grep\
139     $U/_init\
140     $U/_kill\
141     $U/_ln\
142     $U/_ls\
143     $U/_mkdir\
144     $U/_rm\

```

```

145     $U/_sh\
146     $U/_stressfs\
147     $U/_usertests\
148     $U/_grind\
149     $U/_wc\
150     $U/_zombie\
151     $U/_logstress\
152     $U/_forphan\
153     $U/_dorphan\
154     $U/_sync\
155     $U/_peterson\
156     | $U/_prodcons\
157     $U/_readwrite\
158     | $U/_dining\
159
160 fs.img: mkfs/mkfs README $(UPROGS)
161     mkfs/mkfs fs.img README $(UPROGS)
162
163 -include kernel/*.d user/*.d
164
165 clean:
166     rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \
167     */*.o */*.d */*.asm */*.sym \
168     $K/kernel fs.img \
169     mkfs/mkfs .gdbinit \
170     | $U/usys.S \
171     $(UPROGS)
172

```

```

173 # try to generate a unique GDB port
174 GDBPORT = $(shell expr `id -u` % 5000 + 25000)
175 # QEMU's gdb stub command line changed in 0.11
176 QEMUGDB = $(shell if $(QEMU) -help | grep -q '^~gdb'; \
177     then echo "-gdb tcp:$(GDBPORT)"; \
178     else echo "-s -p $(GDBPORT)"; fi)
179 ifndef CPUS
180     CPUS := 3
181 endif
182
183 QEMUOPTS = -machine virt -bios none -kernel $K/kernel -m 128M -smp $(CPUS) -nographic
184 QEMUOPTS += -global virtio-mmio.force-legacy=false
185 QEMUOPTS += -drive file=fs.img,if=none,format=raw,id=x0
186 QEMUOPTS += -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0
187
188 qemu: check-qemu-version $K/kernel fs.img
189     $(QEMU) $(QEMUOPTS)
190
191 .gdbinit: .gdbinit.tmpl-riscv
192     sed "s/:1234/:$(GDBPORT)/" < $^ > $@
193
194 qemu-gdb: $K/kernel .gdbinit fs.img
195     @echo "**** Now run 'gdb' in another window." 1>&2
196     $(QEMU) $(QEMUOPTS) -s $(QEMUGDB)
197
198 print-gdbport:
199     @echo $(GDBPORT)
200

```

```

201 QEMU_VERSION := $(shell $(QEMU) --version | head -n 1 | sed -E 's/^QEMU emulator version ([0-9]+\.[0-9]+)\.*/\1/')
202 check-qemu-version:
203     @if [ "$$(shell echo "$(QEMU_VERSION) >= $(MIN_QEMU_VERSION)" | bc)" -eq 0 ]; then \
204         echo "ERROR: Need qemu version >= $(MIN_QEMU_VERSION)"; \
205         exit 1; \
206     fi
207
208 .PHONY: fmt
209 fmt:
210     clang-format -i $(wildcard kernel/*.ch] user/*.ch] mkfs/*.c)
211

```